LI.FI

AttestedBridgeFacet (v1.0.0)

Independent Security Review

SECURITY RESEARCHER

Sujith Somraaj (somraajsujith@gmail.com)

REPORT PREPARED BY

Sujith Somraaj

Table of Contents

1 About Researcher **1**

2 Disclaimer **1**

3 Scope **1**

4 Risk classification **1**
- 4.1 Impact . . . 1
- 4.2 Likelihood . . . 2
- 4.3 Action required for severity levels . . . 2

5 Executive Summary **2**

6 Findings **3**
- 6.1 Informational . . . 3
  - 6.1.1 Attester call runs outside the shared reentrancy guard and allows nested dispatch . . . 3
  - 6.1.2 Attester must derive facts the gateway does not guarantee . . . 3
  - 6.1.3 Periphery registry storage layout is duplicated in AttestedBridgeFacet . . . 4
  - 6.1.4 nonReentrant requirement on attestable targets is not enforced on-chain . . . 4
  - 6.1.5 Attestable selectors can be enabled before the Diamond routes them . . . 4

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over $170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Valinor, Kuru, MNX, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols. He has uncovered live bugs in Ethereum, Ripple, Uniswap, Morpho, Injective, Phantom, Cosmos, Google and received bug-bounties from those projects.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

AttestedBridgeFacet	`src/Facets/AttestedBridgeFacet.sol` (v1.0.0)
IAttester	`src/Interfaces/IAttester.sol` (v1.0.0)

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

Rating	Definition
High	leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	global losses <10% or losses to only a subset of users, but still unacceptable.
Low	losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

Rating	Definition
High	almost certain to happen, easy to perform, or not easy but highly incentivized
Medium	only conditionally possible or incentivized, but still relatively likely
Low	requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Severity	Required action
Critical	Must fix as soon as possible (if already deployed)
High	Must fix (before deployment if not already deployed)
Medium	Should fix
Low	Could fix

5 Executive Summary

LI.FI engaged with the researcher to audit the contracts described in section 3 of this document ("scope").

The scope covers `AttestedBridgeFacet` and `IAttester` at commit 54e39d0a of contracts PR 2616.

The review identified **5 issues**.

Project Summary

Project Name	**LI.FI Attested Bridge Facet**
Project Type	Solidity Smart Contracts
Repository	lifinance/contracts
Commit	54e39d0a
Audit Timeline	November 11, 2026 - November 13, 2026
Review Duration	5 hours
Methods	Manual Review
Documentation	High
Test Coverage	Medium-High

Issues Found

Critical Risk	**0**
High Risk	**0**
Medium Risk	**0**
Low Risk	**0**
Gas Optimizations	**0**
Informational	**5**
Total Issues	**5**

6 Findings

6.1 Informational

6.1.1 Attester call runs outside the shared reentrancy guard and allows nested dispatch

CONTEXT: AttestedBridgeFacet.sol#L72-L78, ReentrancyGuard.sol#L30-L36, CATValidatorV1_1.sol#L95-L127

DESCRIPTION: `startBridgeAttested` has no lock of its own. The bridge entrypoint takes and releases the shared `ReentrancyGuard` slot inside its delegatecall, so `IAttester.attest` runs with the slot unentered. A throwaway Foundry test with a guarded target and an attester that recursively calls `startBridgeAttested` completed both bridge calls and attested twice in one transaction. `CATValidatorV1_1` keeps `entryId()` active across its target call and sees no nested `entry`, so its own lock doesn't catch this either.

No direct escalation or loss path: the attester is governance-chosen, recursive calls enter with the attester as `msg.sender`, and each recursive bridge call must still pass validation and bring its own assets. It matters if an honest ledger invokes callback-capable code during `attest`, assumes one gateway frame per entry, or lets a nested call overwrite in-flight facts. I couldn't check the private `lifinance/intent-factory` repo.

RECOMMENDATION: Before rollout, Intent Factory tests should show that `attest` only accepts the intended Diamond, receipts are bound to the active `entryId` with a per-entry maximum, and nested `attest` calls cannot overwrite facts. Document that the shared guard covers only the bridge delegatecall. If that can't be proven, add a gateway-local lock in the `com.lifi.facets.attested_bridge` namespace held across `startBridgeAttested`; it must not reuse `com.lifi.reentrancyguard`, or every guarded entrypoint would revert.

LI.FI: Fixed in 532515f.

RESEARCHER: Verified fix.

6.1.2 Attester must derive facts the gateway does not guarantee

CONTEXT: AttestedBridgeFacet.sol#L72-L78, docs/AttestedBridgeFacet.md, AcrossFacetV4.sol#L130-L149, AcrossFacetV4.sol#L225-L236

DESCRIPTION: The gateway only guarantees that the Diamond executed a facet call with exactly these bytes and then called the registered attester. Everything else has to be established by the Intent Factory ledger and facts adapter:

- Receipt uniqueness: two calls can carry byte-identical `_facetCalldata`, so a receipt keyed on its hash alone collapses or replays executions.
- Caller and value: the attester sees the Diamond as `msg.sender` with zero `msg.value`; nothing about the solver or outer value is forwarded.
- Swap entrypoints: `_bridgeData.minAmount` is a floor that gets overwritten in memory with the realized output, and Across V4 also rewrites `AcrossV4Data.outputAmount` via `outputAmountMultiplier`. The attester sees the original calldata.
- Across asset fields: `_bridgeData.sendingAssetId`, the final swap's `receivingAssetId` and `_acrossData.sendingAssetId` are not required to match.
- ABI decoding: a facet can succeed with an undecoded malformed dynamic field, and trailing bytes change the calldata hash without changing semantics (confirmed with a throwaway test).

RECOMMENDATION: No facet change. Before rollout, add ledger/adapter tests for each point above (receipts bound to the active validator entry, `_attesterData` cannot override trusted facts, floor/multiplier semantics, Across field consistency, malformed calldata fails closed, non-canonical encodings handled) and document the list in `docs/AttestedBridgeFacet.md`.

LI.FI: Fixed in 26b7848.

RESEARCHER: Verified fix.

6.1.3 Periphery registry storage layout is duplicated in AttestedBridgeFacet

CONTEXT: AttestedBridgeFacet.sol#L23-L35, PeripheryRegistryFacet.sol#L13-L20

DESCRIPTION: `AttestedBridgeFacet` re-declares the `com.lifi.facets.periphery_registry` namespace and its `mapping(string => address) contracts` field instead of sharing them with `PeripheryRegistryFacet`. The layout is correct today and pinned by `test_BridgeAttesterSlotMatchesTheRegistryFacet` and the Base fork test `test_LiveRegistryWritesTheSlotTheGatewayReads`. Any migration that would break the gateway's read would already corrupt existing registry state.

RECOMMENDATION: Maintainability only. Move the namespace and struct into a shared library used by both facets, or add a `@dev` note on `PeripheryRegistryFacet.Storage` naming `AttestedBridgeFacet` as a direct reader.

LI.FI: Acknowledged - the duplicated layout is pinned by a unit slot test and a fork test against the live registry, and that layout can't change without corrupting existing registrations.

RESEARCHER: Acknowledged.

6.1.4 nonReentrant requirement on attestable targets is not enforced on-chain

CONTEXT: AttestedBridgeFacet.sol#L13, AttestedBridgeFacet.sol#L81-L84, AttestableSelectorsConfig.t.sol#L15-L56

DESCRIPTION: The gateway relies on the dispatched entrypoint taking the shared `com.lifi.reentrancyguard` slot, but nothing verifies the target carries `nonReentrant`. The config test proves which facet owns each signature and the health check proves routing; neither checks the modifier. I verified via the compiler AST that all six selectors in `config/global.json` are `nonReentrant`, so there is no current violation.

An unguarded target would be reachable directly anyway; the added risk is attesting a call that re-entered. Reading the shared guard slot at gateway entry would not enforce the rule, since an unguarded target never enters it.

RECOMMENDATION: Add a CI assertion (source or AST) that every configured attestable signature is `nonReentrant`, or pin selectors to approved code hashes. Re-review the modifier, calldata semantics and facts adapter whenever an enabled facet is replaced.

LI.FI: Acknowledged - the nonReentrant requirement is documented (facet NatSpec, setAttestableSelector @dev, docs)

RESEARCHER: Acknowledged.

6.1.5 Attestable selectors can be enabled before the Diamond routes them

CONTEXT: AttestedBridgeFacet.sol#L86-L96, healthCheckInvariants.ts#L1510-L1516

DESCRIPTION: `setAttestableSelector(_selector, true)` doesn't check that the Diamond routes `_selector`, and the flag isn't enumerable. A quick Foundry test confirmed the sequence: the owner enables an unrouted selector, dispatch reverts with `FunctionDoesNotExist`, a later diamond cut adds the route, and dispatch succeeds with no further setter call. The health check only probes routed and configured selectors, so the stale flag is invisible until the selector gets routed.

Not a privilege bypass: both steps are governed, dispatch fails closed until routed, and the health check flags the mismatch once it is. `attestableSelectors.networks` is empty at `54e39d0a`.

RECOMMENDATION: Require a non-zero route when enabling (keep disabling unconditional). To also catch flags that go stale after a route removal, reconcile the full set from `AttestableSelectorSet` logs or make the set enumerable.

LI.FI: Fixed in 30ec880.

RESEARCHER: Verified fix.